



DL-Steg: a deep learning-based steganography model for improving image security

Ahd Aljarf¹ · Yasmeen Rashidi¹

Received: 12 July 2025 / Accepted: 15 November 2025
© The Author(s) 2025

Abstract

The increasing social media and internet usage and augmentation of users are passing through an era of security risk as an increased concern. Steganography is the method of hiding secret messages inside digital mediums for the secure transmission of the messages. Steganalysis refers to the approach of detecting the presence of hidden information and retrieving it. An appropriate deep learning (DL)-assisted image steganography system should allow hiding and retrieval of hidden images with high accuracy and robustness, ensuring minimal changes in the quality of the cover image and security against detection. In this research work, the proposed framework is DL-based steganography framework integrating a Stacked Autoencoder (SAE) with Long Short-Term Memory (LSTM) networks for compressing, encoding, and embedding secret images into cover images in a secured manner. The evaluation is done on the Classified ImageNet dataset (a pre-processed version of the ImageNet dataset available on Kaggle where images are organized by class and resized for consistency in size). In the proposed method, DL-Steg initially preprocessed the cover and secret images. The encrypting of the secret image, after preprocessing, uses the Elliptic Curve Cryptography (ECC) technique, thus considerably maximizing its security. The secret image encrypted is given as the input to the SAE, which compresses and encodes the data efficiently. The LSTM was implemented to capture sequential dependencies in the image. The encoded secret image was fused with the cover images to produce a container image that can be securely stored or transmitted. These processes are reversed for the secret image's decryption. DL-Steg has a maximum peak to signal noise ratio (PSNR) of 45.5940 and a structural similarity index measure (SSIM) of 0.9877, indicating better image quality and similarity and has a minimum mean squared error (MSE) of 0.143 and a good payload capacity of 24 bpp. The overall research model excelled against all the compared models with better results.

Keywords Steganography · Deep learning · Elliptic curve cryptography · SAE · LSTM · Encryption · Decryption

1 Introduction

Securing digital data has always been at the top of the list in today's world. Large volumes of digital media, which contain audio, video, or more simply, an image, are uploaded on the internet every day. These digital media may potentially carry concealed messages that are in plain sight via a most common technique called "information hiding". This technique allows a person to embed some information into digital data, which leaves the digital data unstressed

by the existence of the hidden message. Therefore, people have no idea about the presence of a hidden message inside any digital medium [1]. Digital data security is assured by steganography and related information hiding methods. Steganography embedding is a well-established and ever-expanding area of science concerned with information hiding [2]. The name steganography comes from the Greek words "steganos", meaning "covered", and "graphein", meaning "writing". While steganography seems to be a method of hiding information within digital data, The input data is called the cover object, while the output is the stego object that contains the secret message [3].

Industrial steganography is used to regulate unauthorized copying of digital data. Copyright societies achieve this by manipulating digital material in such a way that the change is almost invisible to the human eye. The idea is to either

✉ Ahd Aljarf
amjarf@uqu.edu.sa

¹ Cyber Security Department, College of Computing, Umm Al-Qura University, Mecca, Saudi Arabia

assert ownership of the image or to trace recipients of the image. At a strategic military level, this technology has been employed to secretly convey crucial communications without being detected by the opposing party. There is a belief that steganography may have been utilized in the communication methods of criminal organizations and terrorists [4].

Significant differences exist between steganography and other techniques used to embed information. The primary distinction between steganography and cryptography lies in the fact that cryptography conceals the content of communication, whereas steganography conceals the existence of the message [5]. Due to the growing prevalence and utilization of steganography methods, it is imperative to identify concealed messages [6].

As shown in Fig. 1, steganalysis is a method used to determine if a message has been concealed using steganography inside a specific medium. It may be classified into two types: passive and active. The hidden information in the spatial domain is inserted directly by modifying the pixel values in the original image. Conversely, the hidden information in the transformed domain is inserted inside the coefficients of the original image. Passive steganalysis is utilized to identify the existence of hidden messages, whereas active steganalysis is employed to extract the hidden messages. These techniques may be categorized into spatial and frequency domains [7].

Image steganography is a method used to hide one image within another one. In the field of imaging steganography, the cover image is altered in a manner that conceals the secret data, hence preventing it from being detectable and provoking suspicion, unlike encryption. Conversely, Steganalysis is employed to identify the existence of any concealed message embedded within the image and to retrieve the covert information. The steganalysis technique helps in determining whether an image is stego or not. Apart from categorization of such an image, further examination aims

to locate the precise positioning and content of the secret image embedded inside the host image [8].

Demands of real-time communications must be maintained by instantly assessing hidden information, suspected mostly to be found in images, audio, and video. However, real-time concealment and detection of information present quite an exhausting task because one has to factor in bandwidth, latency, and throughput [9]. Given that image steganalysis can help unveil some very complex yet impactful information hidden inside data types automatically some researchers are of the opinion that combining DL with steganalysis can really yield good results, being able to provide a solution to the dilatory standard approach for image steganalysis [10].

With more data emerging every day, DL has garnered popularity and is extensively implemented in several applications. As a technology, deep learning works in myriad applications-from image recognition to automated speech recognizers, image classifiers, recommendation systems, NLP, and analyzing medical images [11].

1.1 Steganography and deep learning

Among traditional steganographic techniques, the Least Significant Bits (LSB) substitution focuses on changing pixel values of the cover image to embed the secret, transformation domain methods (Discrete Cosine Transform, Discrete Wavelet Transform) tweak frequency components, and the histogram method changes characteristic frequency data. These methods are generally quite simple and computationally efficient, yet they are often presented with the dilemma of having to satisfy three basic constraints: imperceptibility (prevent the cover image from being altered in any perceptible manner), capacity (determine how much data can be hidden), and robustness (determine how well it will withstand attacks, distortion-type compression, noise addition, or transmission).

Steganography through DL works on the premise of remaining hidden by having neural networks determine embedding operations automatically. Convolutional Neural Network (CNN), Autoencoder (AE), Generative Adversarial Network (GAN), and hybrid models learn feature mappings with the goal of making changes inconspicuous and allowing for higher payload capacity and greater resistance to steganalysis. Differently than traditional methods, which engage handcrafted rules, DL-based methods learn and adapt embedding patterns from data and thus can stay ahead of modern attacks of steganalysis that also depend on machine learning, or even DL [11].

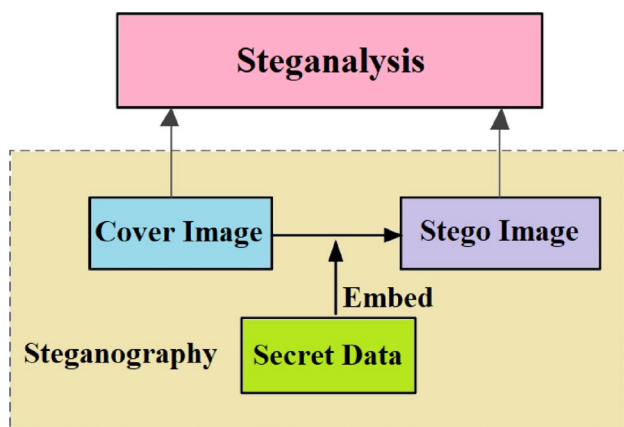


Fig. 1 Steganography and steganalysis process

1.2 Problem statement

In the age of digital information, the security and confidentiality of sensitive images during transmission and storage have gained utmost importance. Unfortunately, classical steganographic techniques often fail to balance payload capacity and imperceptibility, thereby making them vulnerable to detection and attacks. In this study, a new DL-based steganography model that uses a SAE with LSTM networks is proposed with the idea to securely compress, encode, and hide secret images into cover images. The research model is designed to enhance the robustness and security of the steganography method while assuring imperceptibility and detection resistance to fulfill the limitations of the present methods and to further fulfill the increasing demand for secure image communication in various applications.

1.3 Research contributions & objectives

The main contributions to research in this paper are the introduction of a new DL-based steganography model comprising the integration of an SAE with LSTM networks for better compression and encoding of secret images. The model provides for the accurate merging of secret images and their concealment into the cover image. Second, the proposed scheme applies ECC for encryption, thus further enhancing the security of the stego images from detection and unauthorized access. Last but not least, the paper thoroughly assesses the performance using the ImageNet Dataset, which establishes the superiority of the proposed model over existing ones. Together, these contributions move the field of secure image communication forward by resolving in part some fundamental limitations of earlier steganographic methods.

The objectives for this research paper are presented as follows:

- To develop a novel DL-based steganography model to enhance the compression and encoding of secret images within cover images.
- To enhance image security with ECC for ensuring a higher level of security and resistance to detection and unauthorized access.
- To evaluate the proposed model using the ImageNet dataset with assessing metrics such as MSE, PSNR, SSIM.
- To validate the performance of the research model by comparing the results with other current models.
- To address the advantages and limitations of the research model to state it as a secure and efficient solution.

The remaining sections present in the paper are structured as follows: Review of existing research works in the related

works section. Implementation of the proposed DL-Steg research model, including input dataset, preprocessing, ECC, SAE, LSTM, and SAE-LSTM methodology in the proposed methodology section. Results and discussion of the research model with validation in the Experimentation and Analysis section, and lastly, the conclusion and future directions of the work.

2 Related works

This related works explored various DL-based steganography models proposed for enhancing image security. Through analyzing the methodologies, this section highlights a comprehensive analysis of present developments, advantages, and disadvantages in the domain of imaging steganography. A new steganography technique, with large storage capacity, was proposed in [12]. DCT was applied to change secret images, which were then coded using ECC to improve the image's resistance to detection. Thus, SegNet, extraction, and hiding networks further enhanced steganographic capabilities by hiding and retrieving images. Results show the method efficiently distributes all pixels in the image, thereby yielding a relative steganographic capacity of one.

The research described in [13] introduced the SSCNNP framework, a Convolutional Neural-Networks Predictor (CNNP) based on the use of Smooth-Wavelets Transforms (SWT) and Squeeze-Excitation (SE) attention for predicting infrared images. It practically combined CNN and SWT. Formerly, preprocessing half of the input infrared image was utilizing the Super-Resolution CNN (SRCNN) and the SWT. Subsequently, CNNP was utilized to forecast the remaining portion of the infrared image. An attention mechanism was incorporated into the model to enhance the prediction accuracy of CNNP. The findings showed that the framework effectively decreased the error in prediction of the pixels by fully using the characteristics within the pixel in the frequency as well as spatial domains.

A technique to enhance steganography by strengthening the resistance of stego-media against detection using steganalysis was proposed in [14]. The method improved a category of steganographic methods by using a steganographic assisted CNN (SA-CNN). SA-CNN was adaptable in that it allowed the main steganographic algorithm S-UNIWARD to be specifically customized for all the instances of cover material. The findings indicated that the use of SA-CNN during the creation of a stego-image reduced the effectiveness of steganalyzers.

A steganographic approach proposed in [15] that utilized a technique termed adversarial embedding (ADV-EMB) to effectively conceal stego information while deceiving a steganalyzer based on CNN simultaneously. The ADV-EMB

modified the costs of altering imaging components according to the gradients that were back propagated from the targeted CNN steganalyzer. The experiments showed that the steganographic approach improved its security effectiveness against the steganalyzer that was not aware of the adversary's presence by increasing the rate at which it fails to discover the hidden data.

A performance analysis of CNNs using three DL architectures: U-Net, U-Net++, and V-Net was conducted in [16]. The analysis specifically focused on their application in steganography. The encoder architecture produces a stego image that hid the secret images in the cover images. The decoder was designed for effectively amidst the concealment of secret images inside steganographic images. The decoder measured the visual quality of the reconstructed secret image through entropy, SSIM, MSE, PSNR, and *Brisque* Score. This architecture ensures a high payload capacity without compromising the visual quality pertaining to reconstructed and stego images.

The research [17] proposed a simple and uncomplicated architecture towards end-to-end imaging steganography. The concept originated from the DL variant of AE. The model consisted of a preprocessing phase, an extraction module, and an embedding module. The preprocessing stage prepares the input images for the hidden networks that conceal secret images into cover images. The extraction module retrieves hidden images from stego images produced by the embedding module. The model had a higher PSNR value, implying more security and reliability.

In [18], a new steganography system designed for images based on a CNN architecture for encoding and decoding was introduced. A novel loss function was developed for the simultaneous training of the encoder and decoder networks. An exhaustive analysis of the architecture was conducted through various datasets. Architecture attained the highest payload capacity with good PSNR and SSIM values, remodeling entry in this field.

The method in [19] combined steganography, DL, and visual cryptography to address the problem of hiding secret images in cover images. The covert image was the input to an AE, consisting of a decoder and encoder. The image then undergoes cryptography by applying the XOR function with a randomly generated image. The encoded secret image was hidden within the cover images using the LSB method. The secret image was decrypted by reversing every step in the encoding that was applied during its encoding. An assessment of the stego image was done for consistency, with the evaluation of image quality measures indicating the enhancement of security.

The study in [20] presented a new imaging steganographic neural network using an encoder-decoder layout, with the intention of good visual quality. Initially, an

encoder module was implemented using the U-Net + + structure, with the target of attaining near-perfect visual quality. Then, a steganalyzer was inserted into the model by using heuristics to increase the level of security. Finally, with the help of adversarial training, the network model was used to generate stego images. The results showed that the model performs better in visual security and quality.

A detailed study on the famous DL model, namely "SRNet," was undertaken for steganalysis in [21]. It investigates three different variants of SRNet: one being an individual implementation, the second one using TensorFlow, and the third ones using PyTorch. The research illustrated that the choice of DL structure and the parameters used to initialize and optimize the given model have a profound effect on the steganalysis performances. Much importantly, the TensorFlow implementation with an additional dense layer outperforms the other models.

The research in [22] applied a symmetry-adapted DL approach and two-dimensional CNNs to detect concealed patterns in images. SteGuz used three CNNs to perform different stages of the steganographic process on digital covers of images. SteGuz designs a gain function with several image similarity criteria to optimize the imperceptibility of the hiding process. The evaluation of the method was performed with 10 different pairs of cover-secret images. The findings demonstrated that the method surpassed the original method in terms of both the capacity to go unnoticed and the ability to restore.

A high-capacity Invertible Steganography Network (ISN) was designed in [23] for imaging steganography. The steganography and the retrieval of concealed images were considered as two inverse issues related to the modification of image domains. The model has shown extraordinary enhancement of steganography payload capacity, ranking the first in both visual and quantitative evaluations.

An alternative approach was presented in [24] in which a layer-wise adversarial training strategy was attempted to overcome the appearing changes induced by DNN-based imaging steganography. A U-Net type "fountain" architecture with layer-wise adversarial training was used for imaging steganography. Besides the discriminator for the output layer, another sub-network was established that consisted of only one layer and a discriminator placed underneath every layer. This was to capture and utilize the representational abilities of these layers such that the weights of each layer could be updated together with that of their respective discriminators.

In [25], a new technique was proposed in steganography that combines neural networks and fuzzy logic with back-propagation. This technique tries to combine encryption and LSB substitution based on Pixel Value Differencing (PVD) techniques and exploiting modification direction (EMD)

modifications to increase payload capacity, security, and image quality in steganography methods. Thus, the steganographic technique based on fuzzy neural networks opens new opportunities for improving the overall performance.

A Steganography CNN (SteganoCNN) model was proposed in [26] for the dual image embedding problem within a carrier image and the successful reconstruction of the two hidden images. SteganoCNN consists of two modules: encoding and decoding networks. The decoding network contained two extraction networks. At first, the whole network was trained together. The encoding network then automatically encoded the secret images into the carrier images, while the decoding networks reconstructed the two different secret images. SteganoCNN exhibited strong generalization capabilities and hence bigger results.

The research in [27] proposed a deep neural network model able to merge several 2D color images of identical resolutions into cover images of similar resolutions. They applied a deep neural network to hide several hidden images within a cover image. The cover images were also encrypted utilizing ECC for confusion and 3D chaotic maps for diffusion, expressing higher security. The model has shown neural networks' capability to recover a good portion of the hidden images embedded within the original images, with a recovery rate that can go as high as 93%.

Upon reviewing existing research work as shown in Table 1, it is observed that classical steganographic approaches, with their almost trivial computational requirements, tend to have limited payload capacity and thus noticeably degrade image quality for larger volume of data. With the advent of DL technologies, the array of methods that improve on imperceptibility and adaptability were increased but still present some limitations. Most of the methods based on CNNs primarily optimize image reconstruction accuracy without the explicit assumption of sequential dependency in the feature representation; thus, they may limit the embedding quality under diverse image conditions. While AE-based methods were more suitable for compression issues, they are less capable of maintaining long-range dependencies in image structures; other DL-based models, on the other hand, although very powerful, tend to be very unstable to train and need large computational overheads. Finally, the current methods have not incorporated encryption mechanisms, so even if steganalysis succeeds, the hidden data are left vulnerable. The DL-Steg approach, in contrast, aims to circumvent these drawbacks by integrating an SAE with LSTM networks for simultaneous compression and sequential dependency capturing for higher fidelity embedding. The additional use of ECC adds a double layer of security so that exposed hidden data remains safe. This research enables DL-Steg to strike a balance among imperceptibility, payload capacity, and robustness better than any

current approaches, thereby establishing it as a complete image steganography framework.

3 Proposed methodology

A DL-based technique was used in the research to enhance the security and capability of the imaging steganography process. The proposed DL-Steg initiates the preprocessing of both cover and secret images to ensure that they are properly prepared for the following processes. After preprocessing, the secret image is encrypted by an ECC method to enhance its security significantly. After encryption, the secret image is passed onto the SAE to efficiently compress and encode it. To consider the sequential dependencies in the image data, an LSTM model was employed. Further, the LSTM model refines the encoding process. The encoded secret image is then mixed with the cover image to generate the container image. This container is then ready for secure storage or transmission. The container image is processed to extract encoded secret data after transmitted or stored images are received or retrieved. In a similar fashion, this encoded secret data is then decomposed and decoded with the help of the SAE model; thereafter, the decoded secret image is decrypted using ECC technique, yielding the output-the original secret image. This technique provides enhanced security and robustness in image steganography, and at the same time maintains the visual quality of the cover image while enabling the hiding and retrieval of secret images through a reliable process.

3.1 Image dataset

ImageNet is a highly used dataset for classification tasks. It is also used to aid the progress of computer vision research. Researchers employ ImageNet as a dataset for steganalysis by generating stego images from a specific steganography algorithm. ImageNet is valuable for developing novel steganalysis models due to its vast collection of over 14 million colored images of different dimensions. From this ImageNet dataset, a new dataset called Classified ImageNet dataset is derived. Classified ImageNet dataset is a dataset available in the Kaggle repository, which consists of 540,000 high-quality images grouped into 1,000 unique categories (animals, objects, plants, vehicles, etc.). This dataset is collected and used in this research for steganography application. Figure 2 depicts the sample images from the collected dataset, which represents various classes of images present in the dataset. For this research, the dataset is split into 80:20 ratio as training and testing samples [28].

Table 1 Comparative analysis of analyzed works

Ref	Approach used	Application scenario	Advantages	Disadvantages
[12]	DCT, ECC, SegNet	Image steganography with high payload capacity	High PSNR and SSIM values, efficient pixel distribution	Complexity of integrating multiple techniques
[13]	SSCNNP with SWT and SE attention	Infrared image prediction	Reduced prediction error, effective use of pixel characteristics	Complexity in model integration and training
[14]	SA-CNN with S-UNIWARD	Steganography resistant to steganalysis	Reduced effectiveness of steganalyzers	May require customization for different cover materials
[15]	ADV-EMB (Adversarial Embedding)	Hiding stego message and deceiving steganalyzer	Improved security against unaware steganalyzers	Potential vulnerability if adversary is detected
[16]	U-Net, U-Net++, V-Net	Steganography using CNN architectures	Superior performance of U-Net, high visual quality	Higher complexity in training and evaluation
[17]	Deep convolutional autoencoder	End-to-end image steganography	High PSNR value, simple architecture	Limited by the capabilities of the autoencoder
[18]	CNN-based encoder-decoder with novel loss function	Imaging steganography with high payload	High PSNR and SSIM values	Increased training complexity
[19]	Steganography with DL and visual cryptography	Hiding secret image within cover image	Improved security with XOR operation	Complexity in encryption and decryption process
[20]	Encoder-decoder with U-Net ++ and adversarial training	Imaging steganography	Superior visual security and quality	Complexity in model training and adversarial setup
[21]	SRNet for steganalysis	Identifying payloads in digital images	Effective payload detection, impact of initialization	Requires optimization for different payloads
[22]	SteGuz with symmetry-adapted DL	Detecting hidden patterns in images	High imperceptibility and restorability	Complexity in model design and training
[23]	ISN	High-capacity imaging steganography	Exceptional visual quality and high payload	Increased model complexity
[24]	Layer-wise adversarial training with U-Net	Addressing appearance changes in DNN-based steganography	Top results in steganography, effective representation	Complexity in training with adversarial networks
[25]	Neural network with fuzzy logic, PVD, and EMD	Improving payload capacity, security, and image quality	Enhanced overall performance and security	Complexity in integrating multiple techniques
[26]	SteganoCNN	Embedding and reconstructing two hidden images	Strong generalization capabilities, better results	Complexity in training for dual extraction networks
[27]	Deep neural network with ECC and 3D chaotic map	Merging multiple images into a single cover image	High recovery rate of hidden images, enhanced security	Complexity in encryption, diffusion, and model training

3.2 Preprocessing

The images from the dataset have varying dimensions, however they can be reduced to a consistent size before being used as input for the model. The images were obtained from the dataset and pre-processed. The images were rescaled to the resolution of 256×256 pixels and were then normalized by dividing the pixel values by 255. The meaning value of each feature (dimension of images) in the training samples is calculated and then removed from each image. By performing this, the whole training set is transformed into structured data. Therefore, the intensity of the entire training

set is standardized for each dimension. This is accomplished by using Eq. (1).

$$X' = X - \mu \quad (1)$$

In this equation, X' represents the normalized data, X represents the original data, and μ is the mean vector calculated over all the features of X .

The data is initially normalized by the mean and then the standard deviation is calculated and split for each characteristic of the training samples. This process results in the normalization of the mean and variance of the final data.



Fig. 2 Sample images from the dataset

Therefore, the raw data is arranged according on the average and variability of each dimension in the training set. The process of standardizing the input data X is achieved by utilizing Eq. (2).

$$X' = \frac{(X - \mu)}{\sigma} \quad (2)$$

In the given context, X' represents the normalized data, μ represents the mean vector calculated for all the features, and σ represents the standard deviation vector calculated for all the features. Within the framework of Zero Component Analysis (ZCA), the initial step involves normalizing the data (X) by applying feature scaling, which is achieved through the utilization of Eq. (3).

$$X' = \frac{X}{255} \quad (3)$$

First, the data is mean normalized by subtracting the mean value. Then, the singular value decomposition of the covariance matrix of the mean normalized data is computed [29].

3.3 ECC-based encryption

During the process of encryption using ECC, a key is created by selecting an appropriate EC and corresponding parameters. The private key was the confidential key used to encrypt, whereas the public key was derived from the selected EC and the private key was calculated. Furthermore,

during the Conversion and Padding phase of images, the image undergoes a conversion process to transform it into a binary or numerical form. It is crucial to do this step since ECC specifically works with numerical data. Padding is used, if needed, to make sure that the image aligns with the field size of the curve or encrypting specifications. The points compression step was an optional phase where points of the images could be compressed with approaches that minimize the amount of data within encrypted images [12].

In encryption, the images were partitioned into multiple blocks, if required, to enable simultaneous processing. An ephemeral private key (nonce) was selected randomly for each block. The ephemeral public key for the chosen nonce was calculated. The ECC operation involves multiplying the ephemeral public key with the public key of the receiver. This leads to the establishment of a common secret point on the EC. Next, the encrypted block is obtained by performing the XOR operation between the shared secret points and the image blocks. During the decryption stage, the recipient can decode the encrypted image blocks using the private keys that match the public keys utilized for encryptions. The encryption procedure was repeated for all the blocks to get the real image.

The ECC encryption algorithm utilizes the “brainpoolP256r1” curve, which is a 256-bit prime field Weierstrass curve. This curve is imported utilizing the `get_curve` function of registry. The equation representing the curve is given as follows.

$$y^2 = x^3 + ax + b \pmod{p} \quad (4)$$

The finite field comprises a group of integer coordinates $\{x, y\}$ that form an elliptic curve. The values of x and y are restricted to be between 0 and p , inclusive. The constants are expressed in hexadecimal form and can be converted to binary form, which consists of 256 bits. Firstly, the sender must create a private key. Once the private key is produced, the public key is obtained by multiplying the private key value by G . The multiplication of the public and private key yields a shared key, which is utilized by both the sender and recipient, but computed using the public and private keys. The shared key will be represented by the Eq. (4), where the values of x and y are necessary for encryption. The (x, y) values that constitute the point on the Elliptic Curve are transformed into a 256-bit secret key by the process of hashing and concatenating the x and y coordinates. The message is subsequently encrypted using a secret key consisting of 256 bits (equivalent to 32 bytes). To decode, both the private key and the shared key are necessary. The shared key is derived from the product of the receiver's private key and the sender's public key. The shared key will be a 256-bit Elliptic Curve point that will be utilized for decryption [27].

3.4 Stacked autoencoder

An autoencoder is a type of neural network that is designed to learn and extract characteristics from data. Encoding and decoding are the two distinct steps of the operation. During the encoding step, the input data is transformed into a lower-dimensional representation space to extract the most important feature. This feature is then mapped back to the original input space during the decoding phase. The hidden layer is called a bottleneck because the autoencoder is often used for compression purposes. The stacked auto-encoder utilizes the sigmoid and ReLu activation functions. The reconstruction error is reduced when comparing the input and output data, indicating that the autoencoders are effectively learning important elements within the data. The

number of neurons in the output layer of autoencoders is equal to the number of neurons in the input layer. SAE is created by stacking many layers of encoding followed by an output layer of decoding. A stacked autoencoder with two encoding and decoding layers is utilized. Autoencoders are typically trained as a component of a larger model where the input is duplicated. The architecture of the autoencoder model is designed with a restricted bottleneck at the middle, when the input is reconstructed. The model produces a vector of a predetermined length that contains a condensed representation of the input data at a certain point in the model. The structure of SAE consists of several concealed layers, enabling it to effectively represent intricate high-dimensional functions and extract valuable data.

Figure 3 illustrates the structure of the SAE. In the architecture, encoders and decoders are deployed in pairs to map the original features efficiently into reduced features. SAE is thus very useful in reducing the dimensions of data, to make it easy for a classifier to employ and for a machine to act on the text data [30].

3.5 LSTM neural network

Recently, RNN has attracted increased attention for solving prediction problems due to developments brought about with Deep Learning, higher computational capabilities, and because of the inability of standard ML approaches to adequately appreciate the underlying relationships. An RNN retains information in its memory for a very short span based on the kinetics of the iterative process within hidden layers. This mechanism stores some contextual information. And due to exploding and vanishing gradient problems, no long-term memory is offered by RNNs. Henceforth, LSTM was introduced and became a solution widely applied in various domains. LSTM is a type of RNN architecture that enhances the memory capacity of input by including memory cells into the hidden layer. Cell-to-cell communication inside the hidden layer occurs through a sequence of programmed gates, including inputs, outputs, and forget gates. The LSTM model could preserve the cell states by utilizing its gate mechanism. This allows it to effectively address issues related to long-term and short-term memory dependencies, hence preventing challenges such as explosion and vanishing gradients. Figure 4 represents the LSTM architecture, specifically built to represent sequential input [31].

Specifically, the diagram displays four gates, namely input, change, output, and forget. Given a sequence of inputs $\{x_1, x_2, \dots, x_n\}$, where $x_t \in R^{k \times 1}$ represents the data. The - memory cell was updated by three gates: an input gate i_t , a forget-gate f_t , and a change gate $\tilde{c}_t$. The hidden state h_t was modified by including the o_t - output gate and the c_t . The layers and gates calculate the subsequent operations:

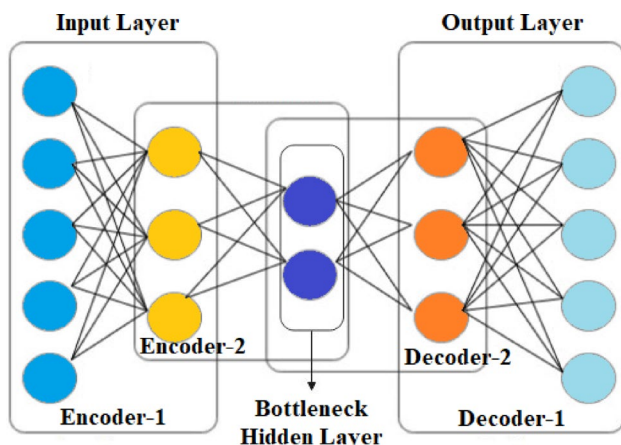


Fig. 3 SAE architecture

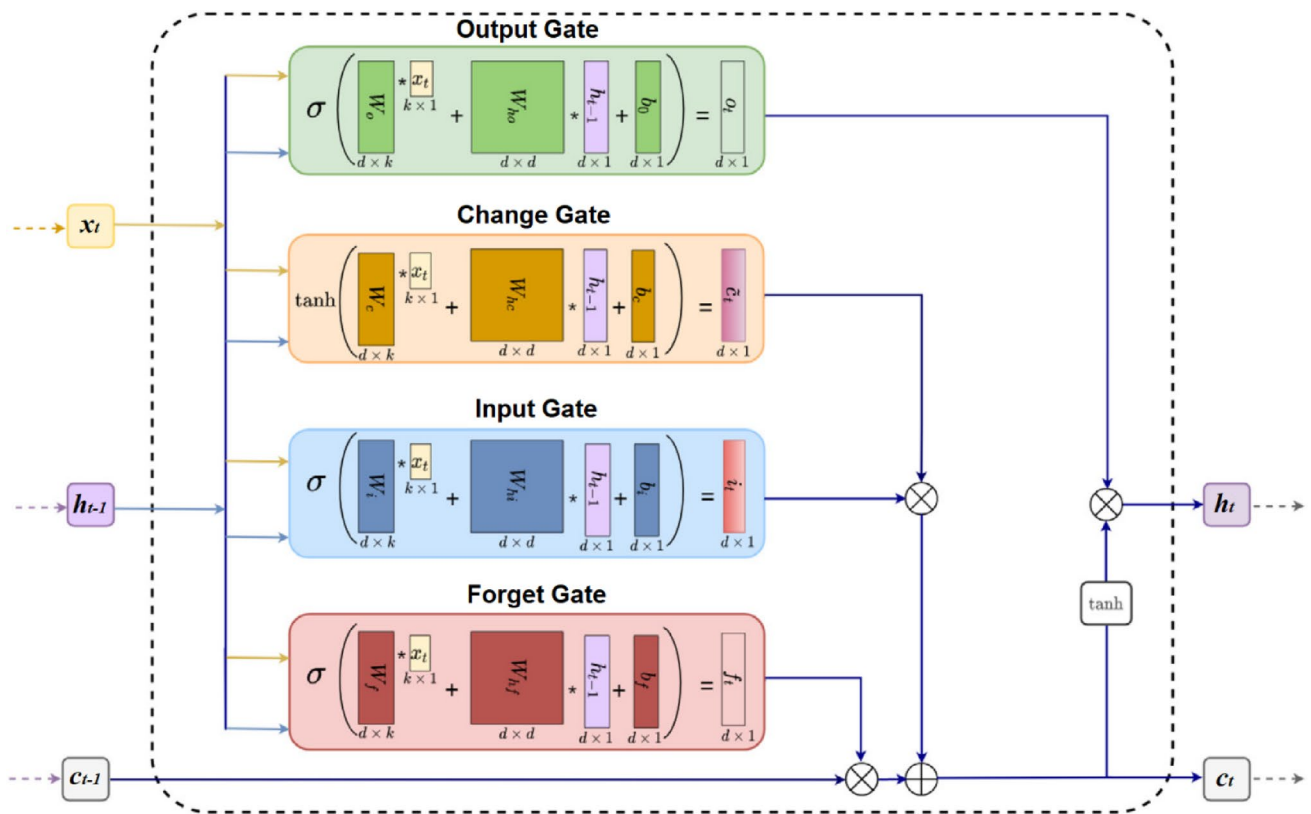


Fig. 4 LSTM architecture

$$i_t = \sigma (W_i x_t + W_{hi} h_{t-1} + b_i) \quad (5)$$

$$h_t = o_t \otimes \tanh(c_t) \quad (6)$$

$$f_t = \sigma (W_f x_t + W_{hf} h_{t-1} + b_f) \quad (7)$$

$$c_t = f_t \otimes c_{t-1} + i_t \otimes \tilde{c}_t \quad (8)$$

$$\tilde{c}_t = \tanh(W_c x_t + W_{hc} h_{t-1} + b_c) \quad (9)$$

$$o_t = \sigma (W_o x_t + W_{ho} h_{t-1} + b_o) \quad (10)$$

The variables $\tanh$ and σ indicate the hyperbolic tangent and sigmoid functions, accordingly. The $\otimes$ operator denotes the elements-wise products. $W \in R^{d \times k}$ and $W_h \in R^{d \times d}$ were matrices of weight, whereas $b \in R^{d \times 1}$ represents bias vectors. Furthermore,, and represent the size of the sequence, the hidden size, and the total features, correspondingly. The cell of LSTM incorporates 3 distinct bits of data: the present input sequences x_t , the previous cell's short-term memory h_{t-1} , and the prior cell state's long-term memory c_{t-1} . The forget gate receives input from h_{t-1} and x_t and produces an outcome ranging from zero to one utilizing the layer of sigmoid. It hence decides which data to exclude from the earlier state of cell c_{t-1} . If

the value was one, it saves every data in the cell, but a zero value causes it to erase every data from the earlier state of cell. In a similar way, the input gates determine the specific data that must be upgraded based on the change gates. The output gates determine the specific data to be selected as the result from the current cell states.

Traditionally, LSTM networks are used with sequential data such as text or speech. However, LSTMs are advantageous in modeling sequential spatial patterns in an image, if the spatial structure is interpreted as a sequence, thereby transforming it from a spatial problem into a temporal domain. In the proposed DL-Steg framework, encoded feature maps from SAE are reshaped into ordered sequences of either patches or pixel vectors. This allows the LSTM network to learn the dependencies between spatially distant regions of an image, which is difficult to achieve using traditional convolutional filters or autoencoders. Using memory cells and gating mechanisms, the LSTMs remain capable of maintaining contextual relationships between image regions sequentially ordered in time, ultimately enhancing the embedding and retrieval fidelity of secret images.

There are two main reasons for the inclusion of LSTM: (i) to boost the representation power of the autoencoder by modeling long-range correlation important for imperceptibility, and (ii) to ensure more uniform distribution of

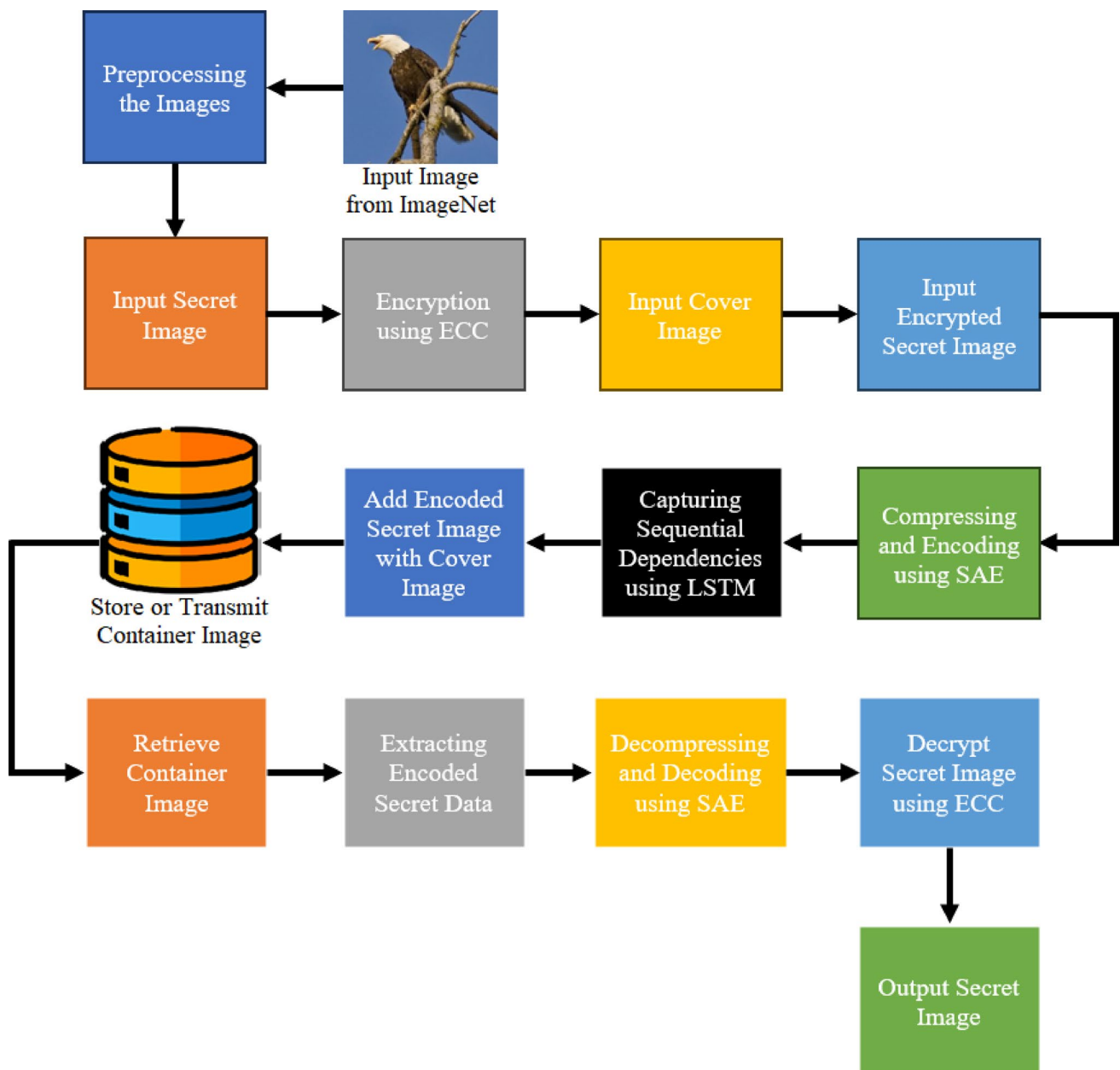


Fig. 5 Workflow of the proposed model

embedding patterns on the cover image, thus providing a stronger resistance to steganalysis. LSTM, being sequence-oriented, can then be adapted to consider spatial sequences within images: this fits neatly as an additional module for SAE in steganographic embedding.

3.6 Proposed SAE-LSTM methodology

During the encoding phase, the SAE reduces the input image dimensionality, which effectively captures its essential features while discarding redundant information. This compressed representation facilitates the covering of the

secret images into the cover images with minimal distortion. Furthermore, in the decoding process, the SAE reconstructs the images from their compressed forms; thus, the secret image retrieved will be near to the original counterpart. These capacities of the SAE enable the efficient use of image steganography, whereby data are compressed and reconstructed simultaneously. Figure 5 depicts the workflow of the research model.

In this research LSTM model is applied to analyze and improve the encoded representation of secret images so that any sequential information contained by image data was preserved and modelled accurately. An LSTM network

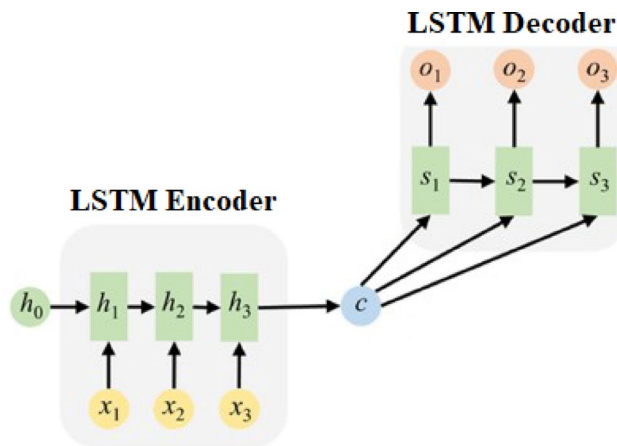


Fig. 6 Structure of encoder-decoder LSTM

Table 2 Hyperparameter details [30]

Hyperparameter	Value	Description
Learning Rate	0.001	Step size used in the optimization process
$\beta 1$	0.9	Exponential decay rate for the first moment estimates
$\beta 2$	0.999	Exponential decay rate for the second moment estimates
Epsilon	1e-7	Small constant to prevent division by zero
Batch Size	64	Number of samples per gradient update
Number of Epochs	100	Number of iterations over the entire dataset
LSTM Units	32	Number of units in the LSTM layers
Encoder Layers (SAE)	3	Number of layers in the Stacked Autoencoder encoder
Decoder Layers (SAE)	3	Number of layers in the Stacked Autoencoder decoder
Dropout Rate	0.5	Fraction of units to drop for regularization
Optimizer	Adam	Algorithm used to optimize the model parameters
Activation Function (SAE)	ReLU	Activation function used in the SAE layers
Activation Function (LSTM)	Tanh	Activation function used in the LSTM layers

retains the temporal coherence of data, thereby contributing to a stronger and more reliable encoding. This is important in the next stages whereby secret images are embedded in cover images; this encoding should, in fact, prevent any foreseen damage to the secret images during extraction. The encoder takes in data and compresses them to generate vectors, which are considered for location-specific context. The decoder then uses this context vector to produce a series of outputs. An input sequence is changed into hidden states by the encoder using RNN or LSTM. The output vectors from the encoder are in correspondence with the last hidden states of the RNN cells. The encoder transmits the context vectors



Fig. 7 Visual representation of DL-Steg

to the decoder. The encoded context vector serves as the initial hidden state for the decoder network, while the output value from the previous time step is used as input for the next LSTM unit to make progressive predictions.

Computationally, the encoder $\varnothing$ was created using the input layers and hidden layers. This encoder compresses the input X from a representation with many dimensions into a representation with fewer dimensions, which is denoted as Z . Meanwhile, the decoder labeled ψ was created using the hidden and output layers. This decoder recreates the input x' . The mathematical representation of these transitions in seq2seq learning involves applying the common neural network function to a sigmoid activation function σ .

$$\varnothing : X \rightarrow Z$$

$$x \mapsto \varnothing(x) = \sigma(Wx + b) = Z \quad (11)$$

$$\psi : Z \rightarrow Z$$

$$z \mapsto \psi(x) = \sigma(\tilde{W}x + \tilde{b}) = x' \quad (12)$$

Here, the variable W was the weight matrices and ' b ' was the bias.

The model consisted of LSTM layers that were added to both the encoder and decoder sections, resulting in a SAE-LSTM. By employing a technique called stacking LSTMs, we might potentially enhance the model's predictive capacity to understand intricate representations of input in the hidden layer by gathering data at several levels [32].

Furthermore, in Fig. 6, x and o represent the inputs and outputs, c represents the context vector of the encoder, and s_t and h_t represent the hidden states in the decoder and encoder, respectively. These are the corresponding elements:

$$h_t = LSTM_{en}(x_t, h_{t-1}) \quad (13)$$

Table 3 Cover and secret images used in experiment

Cover Image	Secret Image
	
Bald Eagle	Binoculars
	
Backpack	African Elephant
	
Aircraft Carrier	African Chameleon
	
Acoustic Guitar	Butterfly
	
Beagle	Acorn

$$h_t = LSTM_{de}(o_{t-1}, s_{t-1}) \quad (14)$$

The encoding LSTM layer computes a context vector, which is then duplicated and transmitted to every decoder unit [32].

The SAE-LSTM model collectively integrates the capabilities of SAE and LSTM networks to improve the process of imaging steganography, refer to Table 2. In this hybrid model, SAE compresses and encodes secret images into a lesser yet information-bearing representation. Such a step ensures that the key features of a secret image are preserved

while its size is dropped for easy embedding into cover images. The LSTM network complements this process by capturing and preserving the sequential dependencies in the encoded data, which proved to be a decisive factor in preserving image integrity and coherence during embedding and retrieval stages. Taking advantage of SAE for feature extraction and compression and LSTM for temporal sequence modeling, the SAE-LSTM model endorsed an efficient, secure and good-quality steganographic operation with minimal loss during the reconstruction of the secret image from the stego image.

4 Experimentation and analysis

4.1 Experimental setup

This section evaluates the performance of the research model through experimental simulations. The experiments were performed using Python 3.6.0 and the APIs of TensorFlow and Keras. The research model was implemented using the Keras API on Google Colaboratory, equipped with a GTX 1050 NVIDIA GPU of 4GB VRAM. The platform for conducting experiments has 16GB RAM, 1 TB HDD, and 256GB SSD. This comprehensive analysis focuses on the efficiency of the proposed novel model, considering various metrics such as MSE, SSIM, PSNR, and payload capacity.

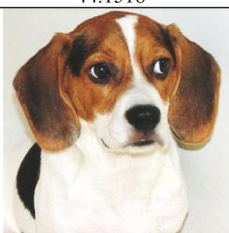
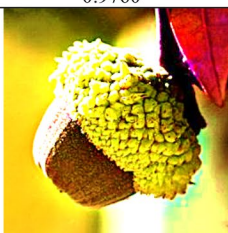
4.2 Performance metrics

Typically, the compressed image must exhibit dissimilarities compared to the actual image. Hence, the PSNR is commonly employed to assess the fidelity of a compressed image in relation to the original image. A greater PSNR value indicates a reduced level of distortion following compression. In imaging steganography, the secret image was embedded as destructive noise, resulting in a degradation of the steganographic images quality. Consequently, a greater PSNR signifies that the hidden picture's noise induced by the secret image is minimal. In simple terms, this standard quantifies the disparity within a carrier and a steganography image. There are two metrics used to evaluate the performance of this research model: PSNR and MSE [12].

$$MSE = \frac{1}{MN} \sum_{i=0}^{M-1} \sum_{j=0}^{N-1} \| Hd(i, j) - Sc(i, j) \|^2 \quad (15)$$

Here, Sc and Hd denote the secret and the host images, and their dimensions are identical. The MSE quantifies the

Table 4 PSNR and SSIM Results on Test Images

Output	PSNR	SSIM
Binocular in Bald Eagle	 43.9849	 0.9857
Elephant in Backpack	 44.2132	 0.9735
Aircraft Carrier in Chameleon	 45.5940	 0.9877
Butterfly in Guitar	 44.1518	 0.9760
Acorn in Beagle	 42.3709	 0.9707

average difference in variance between the original host image and the steganographic image.

$$PSNR = 10 \log_{10} \left(\frac{MAX_I^2}{MSE} \right) \quad (16)$$

Here, MAX_I represents the highest possible value for a pixel in the images. To clarify, the value of MAX_I is equivalent to $2^b - 1$, where b is the total bits per pixel. In grayscale images, the highest possible pixel value is 255.

The PSNR for RGB images, where all the pixels have three parameters of color (R, G, and B), was determined in a similar manner.

SSIM is a metric utilized to quantify the resemblance between two images. The greater the value, the better the outcome. This metric evaluates the images quality by comparing it to a reference image that is free from any distortions. The image under analysis was the stego image, whereas the host image was regarded as the reference image. The formal equation of this metric is given as follows:

$$SSIM(S, C) = \frac{(2\mu_C \mu_S + C_1)(2\sigma_{SC} + C_2)}{(\mu_S^2 + \mu_C^2 + C_1)(\sigma_S^2 + \sigma_C^2 + C_2)} \quad (17)$$

Here, C_1 and C_2 are the variables used to stabilize the division when the denominator is weak. Furthermore, the σ and μ represent the standard deviation and meaning of the variables. The SSIM and PSNR values are derived by comparing the host and the container images, as well as the secret image and the decrypted image.

Furthermore, the capacity of the image information payload in this research model was determined utilizing the following Eq.

$$PC = \left(1 - \frac{\sum_{i=1}^N \sum_{j=1}^M |R_{i,j} - S_{i,j}|}{M \times N} \right) \times 8 \times 3 (bpp) \quad (18)$$

Here, the payload capacity (PC) denotes the quantity of information bits included inside all the pixels. The variables M and N denote the dimensions of the images, specifically their height and width. The secret image was denoted by S while the decrypted image was denoted by R [26]. A qualitative representation related to perception quality is visually depicted in Fig. 7. The results of the proposed DL-Steg model are provided in Fig. 7. The original cover images are shown in first row, and the second row shows the container (stego) images after embedding the secret content. The container images are visually indistinguishable to the cover images, confirming the extremely high imperceptibility of the embedding process. The final row of images are the decoded secret images after extraction and decryption, wherein the images still retain fine structural details as well as semantic content, proving that DL-Steg can perform high-fidelity recovery.

4.3 Performance evaluation

This part presents the results obtained using the proposed DL-Steg model. A series of experiments were performed using the cover images and secret images selected from the input dataset. Table 3 presents the samples of input images utilized as cover and secret images for the experiment.

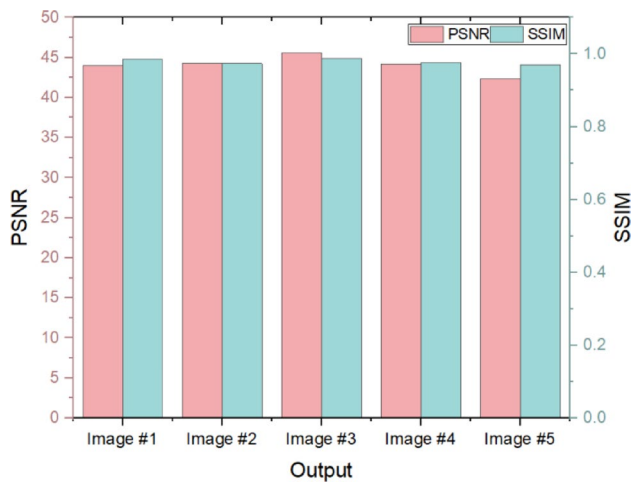


Fig. 8 Graphical plot of PSNR and SSIM results

Table 5 Statistical analysis of aggregated performance

Metric	Mean±Std. Dev.	95% Confidence Interval
PSNR (dB)	45.21±0.67	[45.08, 45.34]
SSIM	0.9854±0.0041	[0.9848, 0.9860]
MSE	0.152±0.013	[0.148, 0.156]

Table 6 Encryption and decryption time

Experiment No.	Encryption Time (sec)	Decryption Time (sec)
1	1.4280	1.2320
2	1.4436	1.2164
3	1.4391	1.1947
4	1.4206	1.2102
5	1.4378	1.1891

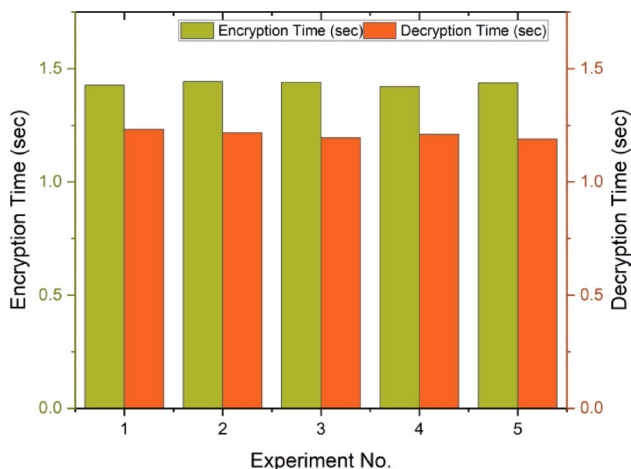


Fig. 9 Graphical plot encryption and decryption time

Based on these images a series of five test experiments were carried out in terms of PSNR, SSIM, encryption time, decryption time, MSE, and payload capacity.

Table 4 presents the PSNR and SSIM results on the test images. The table includes the results on five test images such as Binocular in Bald Eagle, Elephant in Backpack,

Aircraft Carrier in Chameleon, Butterfly in Guitar, and Acorn in Beagle. The Binocular in Bald Eagle image is the representation of the binocular secret image embedded in bald eagle cover image with 43.9849 dB PSNR and 0.9857 SSIM rate. The Elephant in Backpack image represents the elephant as the secret image embedded in backpack cover image with 44.2132 dB PSNR and 0.9735 SSIM. The Aircraft Carrier in Chameleon image represents the aircraft carrier image as the secret image embedded in chameleon cover image with 45.5940 dB PSNR and 0.9877 SSIM. The Butterfly in Guitar image represents the butterfly as the secret image embedded in guitar cover image with 44.1518 dB PSNR and 0.9760 SSIM. Finally, the Acorn in Beagle image represents the acorn as the secret image embedded in beagle cover image with 42.3709 dB PSNR and 0.9707 SSIM. Overall, the Aircraft Carrier in Chameleon image has the best scores of PSNR and SSIM. Figure 8 depicts the graphical plot of the PSNR and SSIM results attained by the research model.

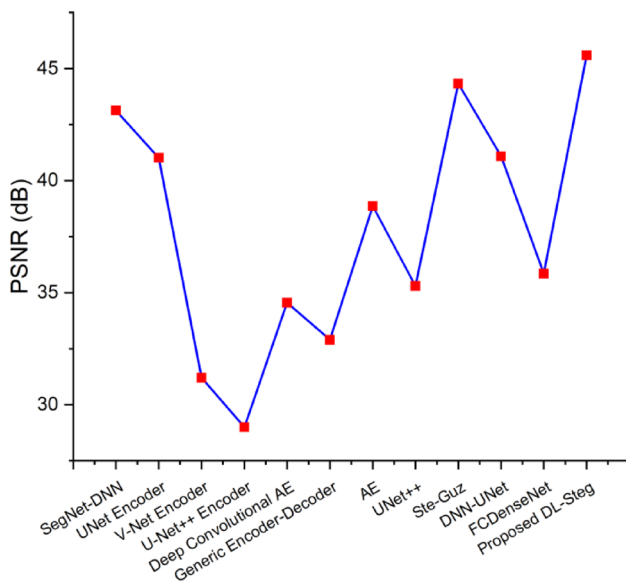
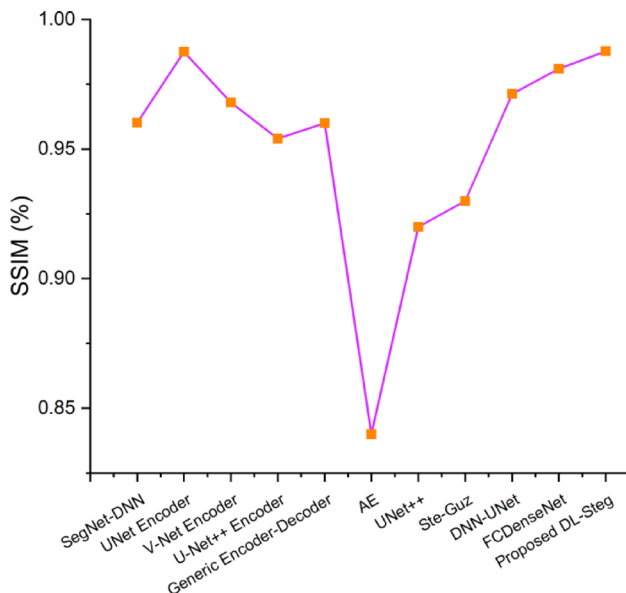
To increase the reliability of evaluation, in Table 5, the DL-Steg model was tested on cover–secret image pairs randomly selected from Classified ImageNet dataset apart from five representative examples previously reported. For all pairs, PSNR, Structural SSIM, and MSE were calculated. These values were then pooled together to report the mean and standard deviation measures, which provide a statistically stronger base from which to evaluate the performance. From the extended results, DL-Steg achieves an average PSNR of 45.21 ± 0.67 dB, an average SSIM of 0.9854 ± 0.0041 , and an average MSE of 0.152 ± 0.013 over the entire evaluation set, with remarkably low deviations, indicating that the model behaves in a stable and consistent manner over different sets of images. To further corroborate the analysis, 95% confidence intervals were calculated for PSNR [45.08, 45.34], SSIM [0.9848, 0.9860], and MSE [0.148, 0.156]. This confirms that the observed improvements in the performance measures are statistically significant and cannot be attributed to random variation.

Table 6 represents the encryption time and decryption time of the experiments carried out in this research. On average the encryption time is 1.4 s and 1.2 s for decryption. Experiment no., 4 has the best encryption time with 1.4206 s and experiment no., 5 has the best decryption time with 1.1891 s. Figure 9 represents the graphical plot of the encryption and decryption time taken by the research model to execute.

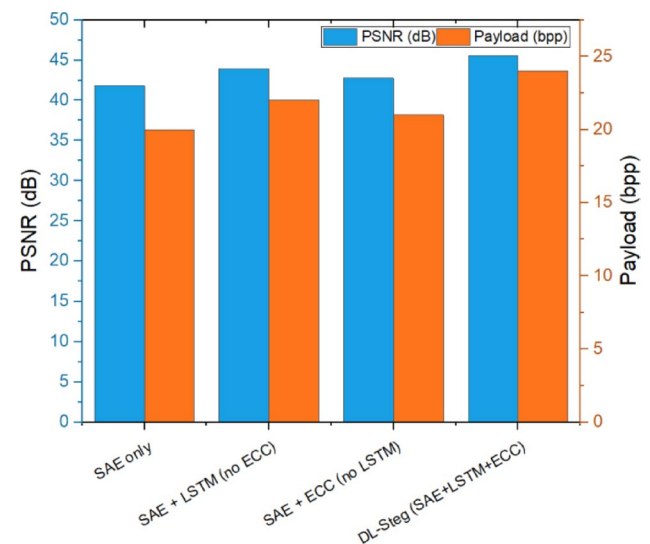
Table 7 presents the results of the proposed research model compared with the current models discussed in the related works section. Most of the compared models utilized the ImageNet dataset and few models utilized images collected from online and COCO dataset. The best scores obtained by the models are compared in this comparison.

Table 7 Comparison of performance analysis

Models	Dataset Used	PSNR	SSIM	MSE	Payload Capacity (bpp)
SegNet-DNN [12]	ImageNet	43.1383	0.9602	NA	23.59
UNet Encoder [16]	Online Images	41.02	0.9875	NA	NA
V-Net Encoder [16]	Online Images	31.20	0.9680	NA	NA
U-Net++ Encoder [16]	Online Images	29	0.9540	NA	NA
Deep Convolutional AE [17]	ImageNet	34.55	NA	0.5197	24
Generic Encoder-Decoder [18]	ImageNet	32.90	0.96	NA	8
AE [19]	Online Image	38.87	0.84	0.845	NA
UNet++ [20]	COCO	35.30	0.92	NA	NA
Ste-Guz [21]	Imagenette	44.33	0.93	NA	24
DNN-UNet [24]	COCO	41.09	0.9713	NA	NA
FCDenseNet [26]	ImageNet	35.852	0.9810	NA	47.925
Proposed DL-Steg	ImageNet	45.5940	0.9877	0.143	24

**Fig. 10** Graphical plot of PSNR comparison**Fig. 11** Graphical plot of SSIM comparison**Table 8** Comparison of DL-Steg model with baseline models

Model	PSNR (dB)	SSIM	MSE	Payload (bpp)
SAE only	41.82	0.9712	0.226	20
SAE+LSTM (no ECC)	43.95	0.9798	0.188	22
SAE+ECC (no LSTM)	42.77	0.9756	0.204	21
DL-Steg (SAE+LSTM+ECC)	45.59	0.9877	0.143	24

**Fig. 12** Graphical plot of PSNR and payload comparison

The proposed DL-Steg model outperforms all the compared current models in terms of evaluated metrics such as PSNR, SSIM, and MSE.

The research model DL-Steg attained its best score of PSNR with 45.5940 dB, which is 1.26% to 16.5% improved than the compared models. Current models such as SegNet-DNN and Ste-Guz have the second-best score of PSNR closer to the proposed model. The least performed model was UNet++ encoder in PSNR. The SSIM score of the research model was 98.77%, which is 0.02% to 14.77% higher than the other models. The UNet encoder model has

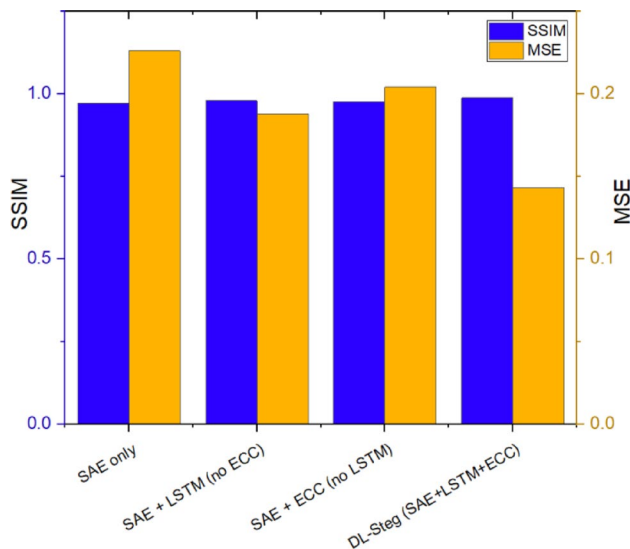


Fig. 13 Graphical Plot of SSIM and MSE Comparison

a very close SSIM rate to the proposed model with 0.9875. The least performed model was AE with 0.84 SSIM. The research model has a low error rate of 0.143 MSE, which is comparatively better than the other models like deep convolutional AE and AE. The research model can carry a payload of 24 bpp, which is quite like what most of the compared models can carry. Figures 10 and 11 represent the graphical plots of PSNR and SSIM.

Overall, the results highlight the performance of the research model in terms of PSNR, SSIM, MSE, and payload capacity for image steganography. This proposed DL-Steg model outperformed all other models with the highest PSNR and SSIM, thus exhibiting better image quality and similarity. Also, it scored the lowest MSE of 0.143 and a high payload capacity of 24 bpp. These parameters showcased the efficiency of the DL-Steg model in ensuring high image quality and robustness while simultaneously providing a fair amount of payload capacity, thereby making it a highly efficient image steganography solution.

To validate the contribution of each component of the framework, an ablation experiment was conducted, in which DL-Steg was compared against three baselines: SAE only, SAE+LSTM (without ECC), and SAE+ECC (without LSTM). As seen in Table 8, the SAE-only system attained rather poor performance (PSNR 41.82 dB, SSIM 0.9712, MSE 0.226) since it was not capable of capturing long-range dependencies and providing any encryption. While LSTM might be assumed to strengthen reconstruction fidelity (PSNR 43.95 dB, SSIM 0.9798, MSE 0.188), it did so by capturing sequential dependencies along the embedding process. Likewise, the addition of ECC with SAE rendered increased security but slightly compromised embedding fidelity (PSNR 42.77 dB, SSIM 0.9756, MSE

0.204). In contrast, DL-Steg (SAE+LSTM+ECC), with the payload of 24 bpp, greatly outperformed all baselines with PSNR 45.59 dB, SSIM 0.9877, and MSE 0.143, proving the combined architecture to be the most ideal in balancing imperceptibility, robustness, and security.

The DL-Steg model proposed for image steganography poses many advantages, the foremost which are greater image quality as reflected by the highest PSNR and SSIM values, and data compression and encoding being most effective with the minimum MSE. It affords a massive payload capacity while retaining the ability to resist detection; hence, it secures and upholds the integrity of the hidden image. As a disadvantage, the model's complexity might escalate computational requirements and training times. Another limitation that arises is the proper balancing of hyperparameters for achieving optimal results across different datasets; this generally involves heavy experimentation and consumption of resources.

Figure 12 represents Graphical Plot of PSNR and Payload Comparison, and Fig. 13 represents Graphical Plot of SSIM and MSE Comparison.

5 Conclusion

This research presents a DL-based steganography model, DL-Steg, developed using DL models such as SAE and LSTM. The main objective of this research is to render a new steganography model for protecting image security with notable improvement in image quality, robustness, and payload amount. This research made use of the classified ImageNet dataset from Kaggle repository, which is collected and served as input to the model for cover and secret images. All images were preprocessed and normalized uniformly. After preprocessing, the secret image was encrypted by applying ECC, thereby improving its security threefold with respect to its concealment. The encrypted secret image is given as input to the SAE with the task of compression and data encoding. To model sequential dependency in the image data, an LSTM model is placed in the encoding path to refine encoding processes further. The encoded secret image is then integrated with the cover image to generate the container image, which is stored or transmitted securely. Once the container image is received or retrieved from transmission or store, it is then processed for extracting coded secret data. This coded secret data undergoes decomposition and decodes the secret image with the SAE model, and finally, the secret image is decrypted via the ECC technique to yield the original secret image. The model attained the highest PSNR of 45.5940 and SSIM of 0.9877, pointing to superb image quality and resemblance. It also attains low MSE of 0.143, along with a reasonable load of opening capacity of

24 bpp. All in all, the proposed research model obtains superiority over all other compared models.

For future work, the efficiency of the SAE-LSTM model will be taken up, paying attention to its robustness, through advanced optimization techniques and adaptive learning rate strategies to reduce computational complexity and training time. Another future parameter would be to consider using other DL architectures and hybrid models that can improve further in terms of performance and security.

Acknowledgements Thanks to Umm Al-Qura University for supporting this research.

Author contributions The paper have been mainly written and revised by two authors. However, the first author has made the main contribution regarding the idea and testing the model.

Funding This research received no external funding.

Data availability The original dataset is available at the Kaggle website. It has been provided within the references list. However, the processed images with this model can be available if it's necessary and upon the reviewer's request.

Declarations

Conflict of interest the authors declare no conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Michaylov KD, Sarmah DK (2025) Steganography and steganalysis for digital image enhanced forensic analysis and recommendations. *J Cyber Secur Technol* 9(1):1–27. <https://doi.org/10.1080/23742917.2024.2304441>
2. Evsutin O, Melman A, Meshcheryakov R (2020) Digital steganography and watermarking for digital images: a review of current research directions. *IEEE Access* 8:166589–166611. <https://doi.org/10.1109/ACCESS.2020.3022779>
3. Eid WM, Alotaibi SS, Alqahtani HM, Saleh SQ (2022) Digital image steganalysis: current methodologies and future challenges. *IEEE Access* 10:92321–92336. <https://doi.org/10.1109/ACCESS.2022.3202905>
4. Reinel TS, Raul RP, Gustavo I (2019) Deep learning applied to steganalysis of digital images: a systematic review. *IEEE Access* 7:68970–68990. <https://doi.org/10.1109/ACCESS.2019.2918086>
5. De La Croix NJ, Ahmad T, Han F (2024) Comprehensive survey on image steganalysis using deep learning. *Array* 100353. <https://doi.org/10.1016/j.array.2024.100353>
6. Wang Z, Byrnes O, Wang H, Sun R, Ma C, Chen H, Xue M (2023) Data hiding with deep learning: a survey unifying digital watermarking and steganography. *IEEE Trans Comput Social Syst* 10(6):2985–2999. <https://doi.org/10.1109/TCSS.2023.3268950>
7. Selvaraj A, Ezhilarasan A, Wellington SLJ, Sam AR (2021) Digital image steganalysis: a survey on paradigm shift from machine learning to deep learning based techniques. *IET Image Proc* 15(2):504–522. <https://doi.org/10.1049/ipr2.12043>
8. Subramanian N, Elharrouss O, Al-Maadeed S, Bouridane A (2021) Image steganography: a review of the recent advances. *IEEE Access* 9:23409–23423. <https://doi.org/10.1109/ACCESS.2021.3053998>
9. Ruan F, Zhang X, Zhu D, Xu Z, Wan S, Qi L (2020) Deep learning for real-time image steganalysis: a survey. *J Real-Time Image Proc* 17:149–160. <https://doi.org/10.1007/s11554-019-00915-5>
10. Ke Y, Liu J, Zhang MQ, Su TT, Yang XY (2018) Steganography security: principle and practice. *IEEE Access* 6:73009–73022. <https://doi.org/10.1109/ACCESS.2018.2881680>
11. Bao Z, Xue R (2021) Survey on deep learning applications in digital image security. *Opt Eng* 60(12):120901–120901. <https://doi.org/10.1117/1.OE.60.12.120901>
12. Duan X, Guo D, Liu N, Li B, Gou M, Qin C (2020) A new high capacity image steganography method combined with image elliptic curve cryptography and deep neural network. *IEEE Access* 8:25777–25788. <https://doi.org/10.1109/ACCESS.2020.2971528>
13. Bai Y, Li L, Lu J, Zhang S, Chu N (2023) A novel steganography method for infrared image based on smooth wavelet transform and convolutional neural network. *Sensors* 23(12):5360. <https://doi.org/10.3390/s23125360>
14. Havard A, Manikas T, Larson EC, Thornton MA (2023) CNN-Assisted Steganography—Integrating machine learning with established steganographic techniques. <https://doi.org/10.48550/arXiv.2304.12503>. arXiv preprint arXiv:2304.12503
15. Tang W, Li B, Tan S, Barni M, Huang J (2019) CNN-based adversarial embedding for image steganography. *IEEE Trans Inf Forensics Secur* 14(8):2074–2087. <https://doi.org/10.1109/TIFS.2019.2891237>
16. Himthani V, Dhaka VS, Kaur M, Rani G, Oza M, Lee HN (2022) Comparative performance assessment of deep learning based image steganography techniques. *Sci Rep* 12(1):16895. <https://doi.org/10.1038/s41598-022-17362-1>
17. Subramanian N, Cheheb I, Elharrouss O, Al-Maadeed S, Bouridane A (2021) End-to-end image steganography using deep convolutional autoencoders. *IEEE Access* 9:135585–135593. <https://doi.org/10.1109/ACCESS.2021.3113953>
18. Rehman R, Rahim, Nadeem S (2018) & S. Hussain End-to-end trained CNN encoders-decoders network for images steganography. In *Proceeding of the European conferences on computer visions (ECCV) workshop*, pp. 1–6. <https://doi.org/10.48550/arXiv.1711.07201>
19. Seuti T, Al Mamun M, Sarowar Sattar AHM (2022) Enhanced steganography technique via visual cryptography and deep learning. In *Proceedings of the International Conference on Big Data, IoT, and Machine Learning: BIM 2021* (pp. 623–636). Springer Singapore. https://doi.org/10.1007/978-981-16-6636-0_47
20. Wang Y, Fu Z, Sun X (2020) High visual quality image steganography based on encoder-decoder model. *J Cybersecur* 2(3):115. <https://doi.org/10.32604/jcs.2020.012275>
21. Kuznetsov A, Luhanko N, Frontoni E, Romeo L, Rosati R (2024) Image steganalysis using deep learning models. *Multimedia*

- Tools Appl 83(16):48607–48630. <https://doi.org/10.1007/s11042-023-17591-0>
22. Khalifa A, Guzman A (2022) Imperceptible image steganography using symmetry-adapted deep learning techniques. *Symmetry* 14(7):1325. <https://doi.org/10.3390/sym14071325>
 23. Lu SP, Wang R, Zhong T, Rosin PL (2021) Large-capacity image steganography based on invertible neural networks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 10816–10825). <https://doi.org/10.1109/CVPR46437.2021.01067>
 24. Chen B, Shi L, Cao Z, Niu S (2023) Layerwise adversarial learning for image steganography. *Electronics* 12(9):2080. <https://doi.org/10.3390/electronics12092080>
 25. Dhawan S, Bhuyan HK, Pani SK, Ravi V, Gupta R, Rana A, Al Mazroa A (2024) Secure and resilient improved image steganography using hybrid fuzzy neural network with fuzzy logic. *Journal of Safety Science and Resilience* 5(1):91–101. <https://doi.org/10.1016/j.jnlssr.2023.12.003>
 26. Duan X, Liu N, Gou M, Wang W, Qin C (2020) Steganocnn: image steganography with generalization ability based on convolutional neural network. *Entropy* 22(10):1140. <https://doi.org/10.3390/e22101140>
 27. Priya S, Abirami SP, Arunkumar B, Mishachandar B (2024) Super-resolution deep neural network (SRDNN) based multi-image steganography for highly secured lossless image transmission. *Sci Rep*. <https://doi.org/10.1038/s41598-024-54839-7>
 28. Classified images dataset (ImageNet 256×256) [Data set]. Kaggle. <https://www.kaggle.com/datasets/dimensions/imagenet-256>
 29. Pal KK, Sudeep KS (2016), May Preprocessing for image classification by convolutional neural networks. In *2016 IEEE International Conference on Recent Trends in Electronics, Information & Communication Technology (RTEICT)* (pp. 1778–1781). IEEE. <https://doi.org/10.1109/RTEICT.2016.7808140>
 30. Kanwal I, Wahid F, Ali S, Rehman AU, Alkhayyat AH, Al-Radaei A (2023) Sentiment analysis using hybrid model of stacked auto-encoder-based feature extraction and long short term memory-based classification approach. *IEEE Access* 11:124181–124197. <https://doi.org/10.1109/ACCESS.2023.3313189>
 31. Ghimire S, Deo RC, Wang H, Al-Musaylh MS, Casillas-Pérez D, Salcedo-Sanz S (2022) Stacked LSTM sequence-to-sequence autoencoder with feature selection for daily solar radiation prediction: a review and new modeling results. *Energies* 15(3):1061. <https://doi.org/10.3390/en15031061>
 32. Yasmifar M (2024) Examining the application of deep LSTM neural networks in steganography of textual information in digital images. *Power Syst Technol* 48(2):292–304. <https://doi.org/10.52783/pst.550>

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.